

REACT / EXPO

Conexión a Backend con Axios

Aprende a comunicar tu aplicación frontend con un servidor usando peticiones HTTP, manejo de errores y buenas prácticas profesionales.

CONCEPTO BASE

¿Qué es un API REST?

Una API REST es el **contrato de comunicación** entre tu frontend y el backend. Usa HTTP como protocolo y JSON como formato de datos.

 Las respuestas siempre llegan en **JSON**, lo que hace que sean fáciles de manipular en JavaScript.

GET

Obtener datos

POST

Crear recurso

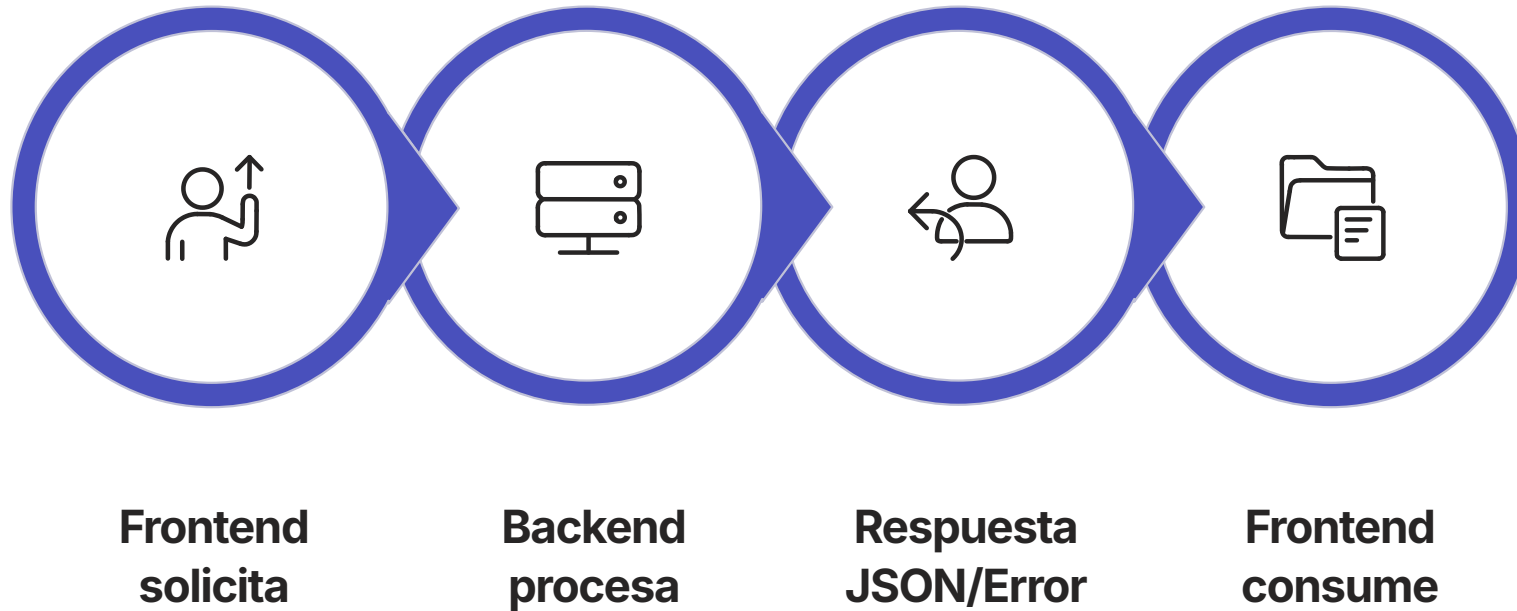
PUT / PATCH

Actualizar

DELETE

Eliminar

¿Cómo funciona la comunicación?



Este ciclo ocurre en cada petición. Entender este flujo es clave para depurar problemas de red y diseñar una buena experiencia de usuario.

REQUISITOS

¿Qué necesitas para conectarte?



URL Base

`https://api.miapp.com`



Endpoints

`/users, /tasks`



Método HTTP

GET, POST, PUT, DELETE



Headers

Autenticación y tipo de contenido



Manejo de errores

Siempre contemplar fallos de red o servidor

HERRAMIENTA PRINCIPAL

¿Qué es Axios?

Axios es una **librería para hacer peticiones HTTP** compatible con React y Expo. Es la alternativa más popular al `fetch` nativo por su sintaxis limpia y sus funcionalidades avanzadas.

Sintaxis clara

Menos boilerplate que `fetch`

JSON automático

No necesitas `.json()` manualmente

Interceptores

Middleware para requests y responses

Mejor manejo de errores

Captura errores HTTP automáticamente

SETUP

Instalación

Elige el gestor de paquetes que uses en tu proyecto:

npm

```
npm install axios
```

yarn

```
yarn add axios
```



Axios funciona sin configuración adicional tanto en **React (web)** como en **Expo (móvil)**.

GET

Primera petición

El patrón básico usa `async/await` con un bloque `try/catch` para capturar errores de red o del servidor:

```
import axios from "axios";

const getUsers = async () => {
  try {
    const response = await axios.get("https://api.example.com/users");
    console.log(response.data);
  } catch (error) {
    console.log(error);
  }
};
```

Estructura del objeto **response**

```
response = {  
  data: {},    // ← los datos del backend  
  status: 200, // ← código HTTP  
  headers: {}, // ← cabeceras de respuesta  
};
```

response.data

El payload principal — lo que normalmente usarás en tu UI.

response.status

Código HTTP: 200 OK, 201 Created, 404 Not Found...

response.headers

Metadatos de la respuesta del servidor.

POST

Petición POST — Crear datos

El segundo argumento de `axios.post()` es el **body** de la petición. Axios lo convierte automáticamente a JSON:

```
const createUser = async () => {  
  try {  
    const response = await axios.post("https://api.example.com/users", {  
      name: "David",  
      email: "david@test.com",  
    });  
    console.log(response.data);  
  } catch (error) {  
    console.log(error);  
  }  
};
```

Uso en un componente React

Combina `useEffect` y `useState` para cargar datos al montar el componente:

```
import { useEffect, useState } from "react";
import axios from "axios";

const Users = () => {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    axios.get("https://api.example.com/users")
      .then(res => setUsers(res.data))
      .catch(err => console.log(err));
  }, []);

  return null; // renderiza tu lista aquí
};
```

📌 El array vacío `[]` en `useEffect` asegura que la petición se haga **solo una vez** al montar el componente.

Hábitos profesionales desde el inicio

1 Usa variables de entorno

Nunca escribas URLs directamente en el código. Usa `.env` para mantener flexibilidad entre ambientes.

3 Maneja errores siempre

Un `catch` vacío puede ocultar bugs graves. Siempre procesa el error.

2 Separa la lógica en servicios

Crea archivos dedicados para las llamadas al API, mantén los componentes limpios.

4 Muestra estado de carga en la UI

Indica al usuario que algo está cargando con un spinner o skeleton.

Instancia de Axios — Base

En lugar de repetir la URL base en cada petición, crea una **instancia centralizada**:

```
import axios from "axios";

const api = axios.create({
  baseURL: "https://api.example.com",
  timeout: 5000,
});

export default api;
```

✔ Ahora usas `api.get("/users")` en lugar de escribir la URL completa cada vez.

Instancia profesional con interceptores

📁 src/services/api.ts — Interceptores para request y response:

```
import axios from "axios";
import Config from "../config";

const api = axios.create({
  baseURL: Config.API_URL,
  timeout: 5000,
});

// REQUEST INTERCEPTOR
api.interceptors.request.use(
  (config) => {
    const newConfig = { ...config };
    newConfig.headers.accept = "application/json";
    const token = import.meta.env.VITE_API_TOKEN;
    if (token) {
      newConfig.headers.Authorization = `Bearer ${token}`;
    }
    return newConfig;
  },
  (error) => Promise.reject(error)
);

// RESPONSE INTERCEPTOR
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) console.warn("No autorizado");
    if (error.response?.status === 500) console.warn("Error del servidor");
    return Promise.reject(error);
  }
);

export default api;
```

Usando la instancia personalizada

Una vez configurada la instancia, todas las peticiones **heredan** la URL base, el timeout, los headers y los interceptores automáticamente.

 Solo necesitas escribir el **path relativo** del endpoint, no la URL completa.

```
import api from "./api";
```

```
// GET → /users  
api.get("/users");
```

```
// POST → /tasks  
api.post("/tasks", {  
  title: "Nueva tarea",  
});
```

¿Por qué usar interceptores?

Los interceptores de Axios te permiten automatizar tareas repetitivas y aplicar lógica global a todas tus peticiones o respuestas, centralizando comportamientos clave de tu aplicación.



Gestión de Tokens

Añade o refresca tokens de autenticación (JWT, OAuth) a cada request de forma transparente.



Headers Consistentes

Establece cabeceras comunes (ej. `Content-Type`, `Accept`) en un solo lugar, evitando repetición.



Registro de Actividad

Implementa un sistema de logging centralizado para monitorizar requests, responses y errores.



Reduce Código Duplicado

Evita escribir la misma lógica una y otra vez en cada llamada API, manteniendo tu código más limpio.

Enviando headers personalizados

Puedes pasar headers en el tercer argumento (o segundo para GET) como parte del objeto de configuración:

```
axios.get("/users", {  
  headers: {  
    Authorization: "Bearer TOKEN",  
    "Content-Type": "application/json",  
  },  
});
```

📌 Con interceptores, **no necesitas repetir** el header de `Authorization` en cada petición — se agrega automáticamente.

Manejo de errores — 3 tipos

```
try {  
  const res = await axios.get("/users");  
} catch (error) {  
  if (error.response) {  
    // backend respondió con error  
  } else if (error.request) {  
    // no hubo respuesta  
  } else {  
    // error general  
  }  
}
```

Problemas frecuentes al conectar

La mayoría de los errores al integrar un API caen en estas categorías. Aprende a identificarlos rápido.

1

CORS

El navegador bloquea la petición. Se resuelve en el **backend**, no en el frontend.

2

localhost en Expo

localhost no funciona en dispositivo físico. Usa tu **IP local**:
192.168.x.x

3

Endpoint incorrecto

Error 404 — ruta mal escrita o método HTTP equivocado.

4

Backend caído

Network Error o ECONNREFUSED — el servidor no responde.

Más errores a tener en cuenta

Timeout

Error `ECONNABORTED` — el backend tardó más que el límite configurado. Ajusta el valor de `timeout` o revisa el servidor.

Datos mal enviados

Error `400 Bad Request` — la estructura del body no coincide con lo que espera el backend.

Headers faltantes

Errores `401 Unauthorized` o `415 Unsupported Media Type` — falta el token o el `Content-Type`.

Token inválido o expirado

Error `401` — el token de autenticación expiró o no es válido. Implementa lógica de refresh.

Variables de entorno mal configuradas

La URL base apunta a un ambiente incorrecto. Verifica tu archivo `.env`.

Checklist cuando no funciona

Cuando te encuentres con errores de conexión, sigue esta lista para identificar rápidamente la causa:

1 ¿Backend corriendo?

Confirma que tu servidor backend esté **activo y escuchando** en el puerto correcto. Un servidor caído es una causa común de `Network Error`.

2 ¿URL correcta?

Revisa la `baseURL` y el path del endpoint. Un error tipográfico o una ruta incorrecta resultará en un `404 Not Found`.

3 ¿Usando localhost en Expo?

Si estás probando en un dispositivo físico con Expo, `localhost` no funciona. Debes usar la **dirección IP local de tu máquina** (ej. `192.168.x.x`).

4 ¿Headers correctos?

Verifica que los headers esenciales como `Authorization` y `Content-Type` se estén enviando correctamente. La instancia de Axios con interceptores debería gestionarlos automáticamente.

5 ¿Token válido?

Si utilizas autenticación, asegúrate de que el token JWT o API key sea **válido y no haya expirado**. Un token inválido suele generar un `401 Unauthorized`.

Expo vs Web — ¿Cambia algo?

React Web

Usa `localhost` normalmente durante el desarrollo. El navegador gestiona las peticiones de red.

Expo (dispositivo físico)

`localhost` apunta al teléfono, **no a tu computadora**. Debes usar tu IP local de red:

```
baseUrl: "http://192.168.1.x:3000"
```



Expo Go en simulador sí puede usar `localhost`, pero en dispositivo real siempre usa la IP.

Organización recomendada del proyecto

Separar las llamadas al API en una carpeta `/services` mantiene los componentes limpios y hace el código reutilizable y fácil de mantener.

- ✓ Esta estructura escala bien desde proyectos pequeños hasta aplicaciones de producción.

1**/services**

`api.ts` — instancia base
`/users/getUsers.ts` — peticiones de usuarios
`/auth/auth.ts` — autenticación

2**/components**

Componentes UI reutilizables

3**/screens**

Pantallas de la aplicación

Estructura completa de /services

Cada dominio tiene su propia carpeta con archivos de responsabilidad única. Todos comparten la misma instancia de api.ts.

```
/services
  api.ts

/users
  getUsers.ts
  getUserById.ts
  createUser.ts
  updateUser.ts
  deleteUser.ts

/auth
  auth.ts
  login.ts
  register.ts
  logout.ts

/tasks
  getTasks.ts
  createTask.ts
  updateTask.ts
  deleteTask.ts
```

1**api.ts**

Instancia base — Axios +
interceptores + headers

2**/users**

CRUD completo de usuarios (5
archivos)

3**/auth**

Token, login, registro y logout (4
archivos)

4**/tasks**

Ejemplo adicional de dominio (4
archivos)

Principios clave de la arquitectura de servicios

Idea clave



Separar por dominio

users, auth, tasks...



Un archivo, una responsabilidad

Cada función hace una sola cosa



Una sola instancia

Todos usan api.ts como base



Regla importante



Componentes

NO usan axios directamente



Componentes

Usan funciones de /services



Beneficio



Código más limpio



Fácil de mantener



Fácil de escalar



Cambios centralizados — solo afecta api.ts

Ejemplo de servicios reutilizables (estructura modular)

Encapsula las llamadas al API en funciones exportables. Los componentes solo importan la función, no saben nada de Axios.

/services/users/getUsers.ts

```
import api from "../api";

export const getUsers = async () => {
  const response = await api.get("/users");
  return response.data;
};
```

/services/users/createUser.ts

```
import api from "../api";

// En lugar de any crear el tipo para data
export const createUser = async (data: any) => {
  const response = await api.post("/users", data);
  return response.data;
};
```

/services/users/getUserById.ts

```
import api from "../api";

export const getUserById = async (id: number) => {
  const response = await api.get(`/users/${id}`);
  return response.data;
};
```

/services/users/deleteUser.ts

```
import api from "../api";

export const deleteUser = async (id: number) => {
  const response = await api.delete(`/users/${id}`);
  return response.data;
};
```

Tipado de servicios con TypeScript

Asegúrate de que tus servicios manejen los datos de forma segura y predecible con tipado estricto.

```
import api from "../api";

interface CreateUserDto {
  name: string;
  email: string;
}

interface User {
  id: number;
  name: string;
  email: string;
}

export const createUser = async (
  data: CreateUserDto
): Promise<User> => {
  const response = await api.post<User>("/users", data);
  return response.data;
};
```

¿Qué significa cada parte?

CreateUserDto

Define la estructura de los datos que **envías** al backend.

Promise<User>

Indica que la función createUser devolverá una promesa que se resolverá con un objeto de tipo User.

User

Define la estructura de los datos que **recibes** del backend.

api.post<User>

Aquí tipas la **respuesta esperada** de Axios, asegurando que el response.data sea de tipo User.

Idea clave

Evitar el uso de any y definir tipos explícitos mejora drásticamente la **claridad y seguridad** de tu código.

Beneficios



Autocompletado

Mejora la experiencia de desarrollo con sugerencias de código precisas.



Menos errores

Detecta problemas de tipo en tiempo de desarrollo, no en producción.



Código más mantenible

Facilita la comprensión y modificación de la base de código para todo el equipo.

Uso de servicios en un componente React

Una vez que tus servicios están definidos, integrarlos en los componentes React mantiene tu lógica de presentación separada de la lógica de comunicación con la API.

```
import { useEffect, useState } from "react";
import { getUsers } from "../services/users/getUsers";

const UsersScreen = () => {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    const loadUsers = async () => {
      try {
        const data = await getUsers(); // Llamada al servicio
        setUsers(data);
      } catch (error) {
        console.error(error); // Manejo básico de errores
      }
    };

    loadUsers();
  }, []); // El array vacío asegura que se ejecuta una sola vez al montar

  return (
    <div>
      <h2>Lista de Usuarios</h2>
      <ul>
        {users.map((user: any) => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    </div>
  );
};

export default UsersScreen;
```

En este ejemplo, el componente `UsersScreen` importa la función `getUsers` del servicio, la ejecuta y maneja el estado local (`users`) así como cualquier error. Nunca interactúa directamente con Axios.



Servicios

Son responsables de realizar las peticiones HTTP al backend y encapsular toda la lógica relacionada con la API.



Componentes

Se encargan de manejar el estado local, la presentación de la interfaz de usuario y la gestión de errores a nivel de UI.

Lo que aprendiste hoy

API REST

Comunicación frontend-backend via HTTP + JSON

Axios

Librería que simplifica peticiones y manejo de errores

Interceptores

Middleware para agregar tokens, logs y manejo global de errores

Servicios

Separar la lógica del API para mantener el código limpio y escalable

 Investigación: Qué es **React Query** o **SWR** para cacheo, refetch automático y estado de carga más sofisticado.